DIGITAL DOWNLOAD (PDF)

# DRx LEARNING HUB

Learn • Practice • Succeed

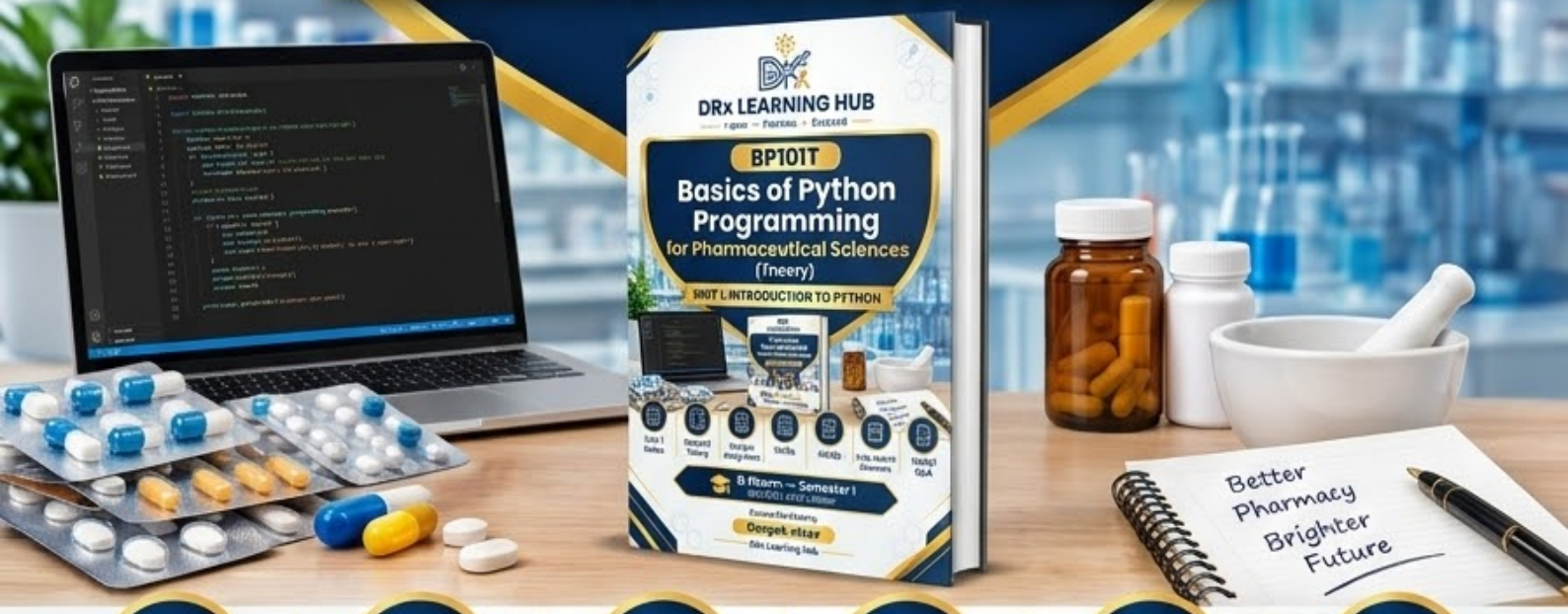
## BP101T

# Basics of Python Programming

## for Pharmaceutical Sciences

## (Theory)

### UNIT I: INTRODUCTION TO PYTHON




- Unit I Notes
- Detailed Theory
- Python Programs
- MCQs
- Long & Short Answers
- Model Q&A

**B.Pharm — Semester I**

NEP 2020 / PCI Curriculum


Compiled & Edited by

**Deepak Jatav**

DRx Learning Hub

# Preface and How to Use These Notes

Dear student, these notes cover **every topic of Unit I** of BP101T (Basics of Python Programming for Pharmaceutical Sciences) exactly as listed in the B.Pharm NEP-2020 syllabus based on the Pharmacy Council of India (PCI) curriculum. They are written in simple English so that even a student who has never seen a programming language can learn from scratch, and they are detailed enough for university examination answers.

Every code example in this book was actually executed and the **green output boxes show the real output**. Please type the programs yourself in your IDE. Programming is learnt by doing, not by reading alone.

## Special boxes used in this book

**DEFINITION**

Formal definitions that you can write directly in the examination.

**EXAM TIP**

Exam tips and likely questions.

**PHARMA CONNECT**

Links between programming and pharmaceutical practice.

**COMMON MISTAKE**

Mistakes that students commonly make.

**REMEMBER**

Key points to remember.

## Unit I syllabus (as prescribed)

| Syllabus point | Covered in |
|---|---|
| Installing Python and an IDE (Jupyter Notebook, PyCharm, VS Code etc.); advantages of IDEs over text editors | Chapters 1, 2 and 3 |
| Python variables and data types (integers, floats, strings, booleans) | Chapter 4 |
| Type casting | Chapter 5 |
| Basic operators (arithmetic, comparison, logical) | Chapter 6 |
| Input and output operations | Chapter 7 |
| Basic string operations and manipulation techniques | Chapter 8 |
| Standard and third-party libraries; installing and uninstalling libraries | Chapter 9 |
| Applications, programs, revision and exam practice | Chapters 10, 11 and 12 |

## Course outcome addressed by Unit I

**REMEMBER - CO1**

Explain the fundamentals of Python programming, including variables, data types, operators, and libraries.

# Table of Contents

**Chapter 1: Introduction to Python Programming . . . 5**

1.1 What is Programming? . . . 5

1.2 What is Python? . . . 5

1.3 Features of Python . . . 5

1.4 How Does Python Run a Program? . . . 6

1.5 Applications of Python in Pharmaceutical Sciences . . . 6

**Chapter 2: Installing Python and an IDE . . . 7**

2.1 Installing Python . . . 7

2.2 Text Editors versus IDEs . . . 8

2.3 Popular Python IDEs . . . 9

**Chapter 3: Your First Programs and Python Syntax . . . 11**

3.1 Your First Python Program . . . 11

3.2 Comments . . . 11

3.3 Indentation . . . 11

3.4 Keywords and Identifiers . . . 11

3.5 Statements and Line Continuation . . . 12

3.6 Common Errors . . . 12

**Chapter 4: Variables and Data Types . . . 13**

4.1 Variables . . . 13

4.2 Data Types . . . 14

4.3 Mutable and Immutable (Introduction) . . . 16

**Chapter 5: Type Casting . . . 17**

5.1 Implicit Type Casting . . . 17

5.2 Explicit Type Casting . . . 17

5.3 Errors in Type Casting . . . 18

5.4 Why Type Casting Matters: the String + Number Problem . . . 18

**Chapter 6: Operators . . . 19**

6.1 Arithmetic Operators . . . 19

6.2 Assignment Operators . . . 20

6.3 Comparison (Relational) Operators . . . 20

6.4 Logical Operators . . . 21

6.5 Other Operators (for awareness) . . . 22

6.6 Pharmaceutical Calculations Using Operators . . . 22

**Chapter 7: Input and Output Operations . . . 23**

7.1 Output using print() . . . 23

7.2 Formatted Output . . . 23
7.3 Input using input() . . . 24
7.4 A Complete Input-Process-Output Program . . . 25
**Chapter 8: Basic String Operations and Manipulation . . . 27**
8.1 Creating Strings . . . 27
8.2 Indexing . . . 27
8.3 Slicing . . . 28
8.4 Concatenation, Repetition and Membership . . . 28
8.5 Immutability . . . 28
8.6 Important String Methods . . . 29
8.7 Useful Built-in Functions with Strings . . . 30
8.8 Strings in Practice: Pharmaceutical Examples . . . 30
8.9 Raw Strings and Special Cases . . . 30
**Chapter 9: Standard and Third-Party Libraries . . . 32**
9.1 Modules, Packages and Libraries . . . 32
9.2 Standard Libraries . . . 32
9.3 Third-Party Libraries . . . 34
9.4 Installing and Uninstalling Libraries with pip . . . 34
**Chapter 10: Integrated Pharmaceutical Programs . . . 37**
Program 1: BMI Calculator (Body Mass Index) . . . 37
Program 2: Dose Calculator by Body Weight . . . 37
Program 3: Creatinine Clearance (Cockcroft-Gault, male) . . . 38
Program 4: Drug Label Reader (strings) . . . 38
Program 5: Expiry Date Calculator (library use) . . . 38
Program 6: Assay Summary using the statistics library . . . 39
**Chapter 11: Unit I Summary, Glossary and Quick Revision . . . 40**
11.1 One-page Summary . . . 40
11.2 Quick Reference Table . . . 40
11.3 Glossary . . . 40
**Chapter 12: Exam Practice Section . . . 42**
12.1 Multiple Choice Questions (30) . . . 42
12.2 Fill in the Blanks . . . 43
12.3 Short Answer Questions with Model Answers (2-3 Marks) . . . 43
12.4 Long Answer Questions (5-10 Marks): Answer Outlines . . . 44
12.5 Predict the Output . . . 45
12.6 Practice Programs . . . 46

# Chapter 1: Introduction to Python Programming

## 1.1 What is Programming?

A **computer program** is a finite set of instructions written in a language that a computer can understand and execute to solve a problem. The person who writes these instructions is called a **programmer**, and the process of writing, testing and correcting them is called **programming** (or coding). Just as a pharmacist follows a prescription step by step, a computer follows a program step by step, exactly as written, without guessing what we "meant".

Programming languages are broadly divided into **low-level languages** (machine language and assembly language, close to hardware, hard for humans) and **high-level languages** (Python, Java, C++, close to English, easy for humans). Python is a high-level language, so a single Python line may represent hundreds of machine instructions.

## 1.2 What is Python?

> **DEFINITION - Python**
>
> Python is a **high-level, interpreted, general-purpose, object-oriented programming language** created by **Guido van Rossum** and first released in **1991**. Its design philosophy emphasises code readability through the use of significant indentation.

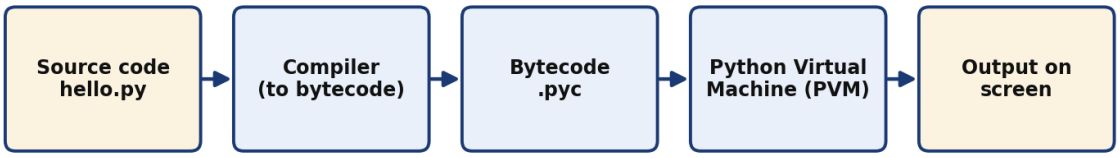
The name "Python" does not come from the snake. Guido van Rossum named it after the British comedy series *Monty Python's Flying Circus*. Python is open source, free to use, and maintained by the Python Software Foundation (PSF). The current major version is Python 3; Python 2 was officially retired in January 2020, so every new program must be written in Python 3.

## 1.3 Features of Python

| Feature | Meaning | Why it matters for a pharmacy student |
|---|---|---|
| Simple and readable | Syntax is close to plain English | You can focus on the pharmaceutical problem, not on complicated syntax |
| Interpreted | Code is executed line by line by the interpreter | Quick testing of a dose formula without a long compile step |
| Dynamically typed | No need to declare the type of a variable | Less code, fewer declarations |
| Free and open source | No licence fee; source code is public | Zero cost for students, colleges and industries |
| Portable (cross-platform) | Runs on Windows, Linux, macOS | Same program works on a lab PC or a laptop |
| Huge library support | Thousands of ready-made libraries | NumPy, Pandas, Matplotlib for data analysis and graphs |
| Object-oriented | Supports classes and objects | Large software can be organised properly |
| Large community | Millions of users, lots of free help | Any error you face has probably been solved online |

## 1.4 How Does Python Run a Program?

When you run a Python file, the interpreter first translates the source code into an intermediate form called **bytecode** (stored in `.pyc` files). The bytecode is then executed by the **Python Virtual Machine (PVM)**. If an error is found, execution stops at that line and an error message (traceback) is shown. This is why Python is called an **interpreted** language.




How Python runs a program: source code is compiled to bytecode and executed by the PVM

*Fig. 1.1: Execution of a Python program*

| Compiler | Interpreter |
|---|---|
| Translates the whole program at once | Translates and executes one statement at a time |
| Errors are shown after full scanning | Stops at the first error found |
| Faster execution (e.g., C, C++) | Slightly slower, but easier to test and debug (e.g., Python) |
| Creates a separate executable file | Does not necessarily create a separate executable file |

## 1.5 Applications of Python in Pharmaceutical Sciences

The PCI NEP-2020 syllabus introduces Python because modern pharmacy is data driven. Python is used wherever numbers, tables and graphs are involved:

- **Pharmacokinetics (PK):** calculating half-life, clearance, AUC; plotting concentration-time curves for oral and IV doses.
- **Pharmacovigilance:** analysing Adverse Drug Reaction (ADR) reports to find patterns and signals.
- **Formulation and dissolution:** comparing dissolution profiles, similarity factor (f2), batch data analysis.
- **Quality control and QA:** statistics on assay results, trend charts, automatic report generation.
- **Drug discovery and cheminformatics:** handling molecular structures, QSAR models, virtual screening (libraries such as RDKit).
- **Hospital and community pharmacy:** dose calculators, BMI and creatinine-clearance tools, inventory tracking.
- **Artificial Intelligence:** machine-learning models for disease prediction, drug-target interaction and clinical-trial data.

> **PHARMA CONNECT - Why this unit matters**
>
> Unit I is the foundation. Without clear understanding of variables, data types, operators, input/output, strings and libraries, Units II to V (control structures, NumPy, Pandas and Matplotlib) cannot be understood.

# Chapter 2: Installing Python and an IDE

## 2.1 Installing Python

Python is downloaded free of cost from the official website **python.org** (Downloads section). Always download the latest stable Python 3 release from the official site only, never from unknown third-party links.

### A. Installing on Windows

1. Open **python.org** and click **Downloads**; the site automatically offers the Windows installer (64-bit) for the latest version.
2. Run the downloaded `.exe` file.
3. **Tick the box "Add python.exe to PATH"** on the first screen (very important; without it the command `python` will not work in Command Prompt).
4. Click **Install Now** and wait for "Setup was successful".
5. Open Command Prompt and verify the installation (see below).

### B. Installing on macOS and Linux

- **macOS:** download the macOS installer (`.pkg`) from python.org and follow the wizard; or use Homebrew: `brew install python`.
- **Linux (Ubuntu/Debian):** Python 3 is usually pre-installed. To install or update: `sudo apt update` then `sudo apt install python3 python3-pip`.

### C. Verifying the installation

**TERMINAL / COMMAND PROMPT**

```
python --version
pip --version
```

On Linux and macOS you may need to type `python3` and `pip3`. A version number such as Python `3.12.x` confirms a successful installation. Typing just `python` opens the interactive prompt >>> (called the **REPL** - Read, Evaluate, Print, Loop), where each line is executed immediately. Use `exit()` to leave it.

### D. Two ways of running Python code

| Mode | How | Best used for |
|---|---|---|
| Interactive mode | Type commands at the >>> prompt; result appears instantly | Quick calculations and testing small ideas |
| Script mode | Save code in a file such as dose.py and run: python dose.py | Complete programs that you want to save and reuse |

### E. The Anaconda alternative

**Anaconda** is a free distribution of Python that installs Python, Jupyter Notebook, Spyder and hundreds of scientific libraries (NumPy, Pandas, Matplotlib, SciPy) together in one go. It is popular in data science and is convenient for students who want everything ready without installing libraries one by one. The disadvantage is a large download size (several GB).

## 2.2 Text Editors versus IDEs

A **text editor** (Notepad, Notepad++, Sublime Text in basic mode) only lets you type and save text. To run the program you must switch to a terminal and run it manually. An **Integrated Development Environment (IDE)** is a software application that combines an editor, a way to run the code, a debugger and many other tools in *one single window*.

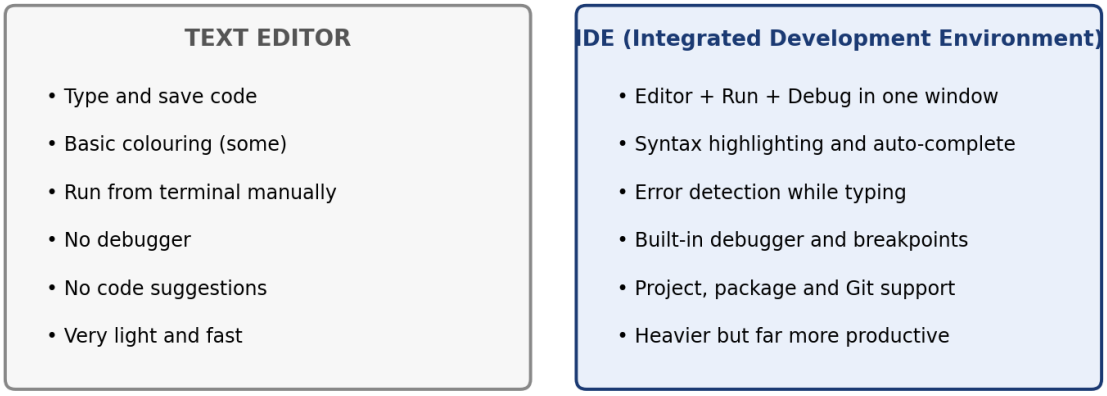



*Fig. 2.1: Text editor versus IDE*

### Advantages of IDEs over text editors (important for exams)

1. **Syntax highlighting:** keywords, strings and numbers are shown in different colours, so mistakes are noticed easily.
2. **Code completion (IntelliSense):** suggests function and variable names while typing, which saves time and reduces spelling mistakes.
3. **Error detection:** underlines syntax errors while you type, before you even run the program.
4. **Built-in debugger:** lets you stop at a line (breakpoint), run line by line and watch variable values.
5. **One-click execution:** run the program with a button or shortcut; the output appears in the same window.
6. **Project management:** organises many files and folders of a project together.
7. **Package/library management:** install and manage libraries and virtual environments from inside the IDE.
8. **Version control integration:** works with Git to track changes.
9. **Refactoring and code formatting:** renames variables safely and arranges code neatly (indentation).
10. **Documentation help:** shows information about a function when you hover over it.

| Point of comparison | Text editor | IDE |
|---|---|---|
| Components | Only writing and saving | Editor + interpreter + debugger + tools |
| Running code | Separate terminal required | Run button inside the window |
| Auto-complete | Absent or very limited | Intelligent suggestions |
| Debugging | Not available | Breakpoints, step-by-step execution |
| Error checking | After running only | While typing |
| Resource use | Light and fast | Heavier, needs more RAM |
| Best for | Quick edits to files | Developing complete programs |

## 2.3 Popular Python IDEs

### 1. Jupyter Notebook

Jupyter Notebook is a **web-browser based interactive environment** in which code, output, text (Markdown), equations and graphs live together in one document called a **notebook** (file extension `.ipynb`). A notebook is made of **cells**. A *code cell* is run with **Shift + Enter** and its output appears just below it. A *Markdown cell* is used to write headings and explanations.

- **Installation:** `pip install notebook` (or install Anaconda, which includes it).
- **Launch:** type `jupyter notebook` in the terminal; the browser opens automatically.
- **Kernel:** the background "engine" that runs your code; "Restart Kernel" clears all variables.
- **Strengths:** ideal for learning, data analysis, graphs and sharing reports; results are visible immediately.
- **Limitations:** not ideal for very large programs; cells can be run out of order and cause confusion.
- **Related tool: Google Colab** is a free cloud-based Jupyter notebook that needs no installation, only a Google account.

### 2. PyCharm

PyCharm is a full-featured IDE made by **JetBrains** exclusively for Python. It comes in two editions: **Community Edition** (free, open source, suitable for students) and **Professional Edition** (paid, with web development and scientific tools). It has very strong code completion, refactoring, a powerful debugger and automatic creation of a **virtual environment** for every new project.

- **Steps:** install PyCharm, click *New Project*, choose the location and interpreter, add a new Python file, and press the green **Run** button.
- **Strengths:** best suited for large projects and professional development.
- **Limitations:** heavy on memory; slow on low-end computers.

### 3. Visual Studio Code (VS Code)

VS Code is a free, lightweight, open-source code editor from **Microsoft**. By itself it is an editor, but after installing the **Python extension** (and the Jupyter extension) from its Extensions marketplace, it behaves like a complete IDE. It supports many languages, has an integrated terminal, Git support and thousands of extensions.

- **Steps:** install VS Code, open the Extensions tab, install "Python", open a folder, create `file.py`, choose the interpreter with *Ctrl+Shift+P → Python: Select Interpreter*, and press the Run button.
- **Strengths:** light, fast, customisable, can also run notebooks.

### 4. Other IDEs worth knowing

- **IDLE:** the simple IDE that comes bundled with Python itself.
- **Spyder:** a scientific IDE (comes with Anaconda) with a variable explorer, popular with researchers.
- **Google Colab:** browser-based; free GPU access; no installation.

| IDE | Type / Maker | Cost | Best use |
|---|---|---|---|
| Jupyter Notebook | Browser notebook (Project Jupyter) | Free | Learning, data analysis, graphs, reports |
| PyCharm | Full IDE (JetBrains) | Community free / Pro paid | Large, professional projects |
| VS Code | Editor + extensions (Microsoft) | Free | General coding, light and flexible |
| IDLE | Basic IDE (bundled with Python) | Free | First steps, tiny programs |

| IDE | Type / Maker | Cost | Best use |
|---|---|---|---|
| Spyder | Scientific IDE | Free | Scientific computing and research |

**EXAM TIP - Likely question**

Q. Define IDE. Write its advantages over text editors. → Write the definition, any 6 to 8 advantages from the list in 2.2 and, if marks are 5 or more, one comparison table plus names of Jupyter, PyCharm and VS Code with one line each.

# Chapter 3: Your First Programs and Python Syntax

## 3.1 Your First Python Program

**PYTHON → OUTPUT shown in green**

```
print("Hello, Pharmacy!")
```

```
Hello, Pharmacy!
```

`print()` is a built-in **function** that displays output on the screen. The text inside the quotation marks is a **string**. Every Python statement ends when the line ends; no semicolon is needed.

## 3.2 Comments

A **comment** is a note for humans that Python ignores. Good programmers comment their code so that they (and others) can understand it later.

**PYTHON → OUTPUT shown in green**

```
# This is a single-line comment
dose = 500   # inline comment: dose in mg

"""
This is a multi-line comment
(technically a docstring-style string)
"""
print(dose)
```

```
500
```

## 3.3 Indentation

Most languages use braces { } to group statements. Python uses **indentation** (spaces at the beginning of a line). The standard is **four spaces**. Wrong indentation causes `IndentationError`. You will use this heavily in Unit II (if, loops, functions).

**PYTHON → OUTPUT shown in green**

```
age = 20
if age >= 18:
    print("Adult dose")      # indented block
    print("Check weight too")
print("Done")                # not indented: runs always
```

```
Adult dose
Check weight too
Done
```

## 3.4 Keywords and Identifiers

**Keywords** are reserved words that have a special meaning in Python and cannot be used as variable names. There are 35 keywords in current Python 3 versions. You can list them yourself:

**PYTHON → OUTPUT shown in green**

```
import keyword
print(len(keyword.kwlist))
print(keyword.kwlist)
```

```
35
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class', 'continue', 'def', 'del',
```

An **identifier** is the name given to a variable, function or other object. The rules are:

- It may contain letters (A-Z, a-z), digits (0-9) and the underscore (_).
- It **cannot start with a digit** (`1dose` is invalid).
- It **cannot contain spaces or special symbols** such as @, #, $, %, -.
- It **cannot be a keyword** (`class`, `if`, `for`...).
- It is **case-sensitive**: `Dose`, `dose` and `DOSE` are three different names.

## 3.5 Statements and Line Continuation

One instruction is called a **statement**. Several statements can be written on one line using a semicolon (not recommended). A very long statement can be continued on the next line using a backslash \ or by placing it inside brackets.

**PYTHON → OUTPUT shown in green**

```
total = 100 + 200 + \
        300
print(total)

bmi_text = ("Body Mass Index "
            "is weight divided by height squared")
print(bmi_text)
```

```
600
Body Mass Index is weight divided by height squared
```

## 3.6 Common Errors

| Error type | Cause | Example |
|---|---|---|
| SyntaxError | Rule of the language broken | `print("Hi" (missing bracket)` |
| IndentationError | Wrong spacing at line start | `Unexpected indent` |
| NameError | Variable used before it is created | `print(dosee)` |
| TypeError | Operation on wrong data types | `"5" + 5` |
| ValueError | Right type, unsuitable value | `int("abc")` |
| ZeroDivisionError | Dividing by zero | `10 / 0` |

**REMEMBER**

Errors are of three kinds: **syntax errors** (found before running), **runtime errors** (found while running, like ZeroDivisionError) and **logical errors** (program runs but gives a wrong answer; hardest to find).

# Chapter 4: Variables and Data Types

## 4.1 Variables

> **DEFINITION - Variable**
>
> A **variable** is a named location in the computer memory used to store a value that can change during the execution of a program. In Python, a variable is created the moment a value is assigned to it using the assignment operator **=**. No separate declaration is needed.

Think of a variable as a labelled bottle in a pharmacy store: the label is the variable name, the content is the value, and you can replace the content later without changing the label.

**PYTHON → OUTPUT shown in green**

```
drug = "Paracetamol"      # string
dose_mg = 500             # integer
price = 12.75             # float
in_stock = True           # boolean
print(drug, dose_mg, price, in_stock)
```

```
Paracetamol 500 12.75 True
```

### Dynamic typing

Python is **dynamically typed**: the type of a variable is decided automatically from the value assigned to it, and the same name can later hold a value of a different type. The functions `type()` and `id()` tell us the type and the memory identity of an object.

**PYTHON → OUTPUT shown in green**

```
x = 10
print(type(x))
x = "ten"
print(type(x))
x = 10.0
print(type(x), id(x) == id(x))
```

```
<class 'int'>
<class 'str'>
<class 'float'> True
```

### Multiple assignment and swapping

**PYTHON → OUTPUT shown in green**

```
a = b = c = 0                # same value to many variables
name, age, weight = "Ravi", 21, 58.5   # different values
print(a, b, c)
print(name, age, weight)

x, y = 5, 9
x, y = y, x                  # swap without a temporary variable
print(x, y)
```

```
0 0 0
Ravi 21 58.5
9 5
```

### Naming conventions (good practice)

- Use meaningful names: `body_weight` is better than `bw` or `x`.
- Use **snake_case** (lowercase words joined by underscores) for variables: `daily_dose_mg`.
- Constants (values that should not change) are written in CAPITAL LETTERS by convention: `AVOGADRO = 6.022e23`. Python has no true constants; this is only a convention.
- Avoid single letters l, O, I because they look like numbers.

## 4.2 Data Types

A **data type** specifies the kind of value a variable holds and the operations that can be performed on it. The four fundamental (basic) types in the syllabus are **integer, float, string and boolean**.

| Data type | Python name | Meaning | Examples |
|---|---|---|---|
| Integer | `int` | Whole numbers, positive, negative or zero; no size limit | 25, -3, 0, 100000 |
| Floating point | `float` | Numbers with a decimal point | 72.5, -0.05, 3.0, 1.5e-3 |
| String | `str` | Sequence of characters in quotes | "Aspirin", '500 mg' |
| Boolean | `bool` | Only two values: True or False | True, False |

### 1. Integer (int)

Integers store whole numbers such as number of tablets, age or batch number. Python integers can be of any size.

**PYTHON → OUTPUT shown in green**

```
tablets = 30
negative = -8
big = 2 ** 100
print(tablets, negative)
print(big)
print(type(big))
```

```
30 -8
1267650600228229401496703205376
<class 'int'>
```

### 2. Floating point (float)

Floats store decimal values such as weight, concentration and price. Very large or small values can be written in **scientific notation**: `1.5e-3` means $1.5 \times 10^{-3} = 0.0015$.

**PYTHON → OUTPUT shown in green**

```
weight = 62.5
conc = 2.5e-3
print(weight, conc)
print(type(weight))
print(0.1 + 0.2)
print(round(0.1 + 0.2, 2))
```

```
62.5 0.0025
<class 'float'>
0.30000000000000004
0.3
```

**COMMON MISTAKE - Floating point precision**

Computers store decimals in binary, so some values (like 0.1) cannot be stored exactly. Hence `0.1 + 0.2` gives `0.30000000000000004`. Use `round(value, digits)` when displaying results such as doses or concentrations.

### 3. String (str)

A string is a sequence of characters enclosed in single quotes, double quotes or triple quotes. Triple quotes allow strings that span many lines. Strings are covered in detail in Chapter 8.

**PYTHON → OUTPUT shown in green**

```
a = "Ibuprofen"
b = 'Tablet IP 400 mg'
c = """Store below 25 C.
Protect from light."""
print(a)
print(b)
print(c)
```

```
Ibuprofen
Tablet IP 400 mg
Store below 25 C.
Protect from light.
```

### 4. Boolean (bool)

A boolean has only two values, `True` and `False` (the first letter must be capital). Booleans are the result of comparisons and are the basis of decision making in Unit II. Internally, True behaves as 1 and False as 0.

**PYTHON → OUTPUT shown in green**

```
is_expired = False
print(is_expired, type(is_expired))
print(10 > 5)
print(True + True)
```

```
False <class 'bool'>
True
2
```

### Other built-in types (for awareness)

- **NoneType:** the special value `None` means "no value" (e.g., a variable not yet filled).
- **complex:** numbers like `3+4j` (rarely used in pharmacy).
- **list, tuple, dictionary, set:** collections of values; Unit III covers lists, tuples and dictionaries.

### Checking type with type() and isinstance()

**PYTHON → OUTPUT shown in green**

```
value = 25.4
print(type(value))
print(isinstance(value, float))
print(isinstance(value, int))
```

```
<class 'float'>
True
False
```

## 4.3 Mutable and Immutable (Introduction)

Integers, floats, strings and booleans are **immutable**: once created, their value cannot be changed in place; assigning a new value creates a new object. Lists and dictionaries (Unit III) are **mutable** and can be changed in place. Remember that *strings are immutable*, a frequent exam point.

> **EXAM TIP**
>
> Always mention **four types: int, float, str, bool** with one example and the `type()` function in any answer on data types. Add "Python is dynamically typed" as a bonus line.

# Chapter 5: Type Casting

> **DEFINITION - Type casting**
>
> **Type casting** (type conversion) is the process of converting a value from one data type to another, for example a string to an integer.

It is needed because `input()` always returns a string. If the user types a dose of 500, Python receives the text "500", which cannot be used for calculation until it is converted to a number.

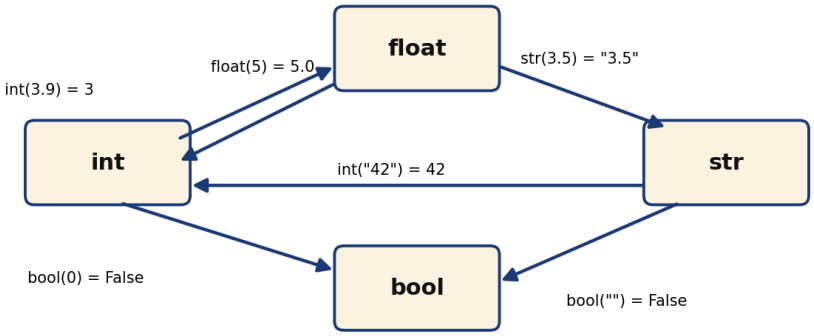



*Fig. 5.1: Conversions between data types*

## 5.1 Implicit Type Casting

Here Python converts the type **automatically**, without being asked, always from a smaller to a wider type to avoid data loss. For example, int + float gives float.

**PYTHON → OUTPUT shown in green**

```
a = 10        # int
b = 2.5       # float
c = a + b     # int is converted to float automatically
print(c, type(c))

print(True + 5, type(True + 5))
```

```
12.5 <class 'float'>
6 <class 'int'>
```

## 5.2 Explicit Type Casting

Here the programmer **forces** the conversion using a built-in function. The main functions are:

| Function | Converts to | Example | Result |
|---|---|---|---|
| `int()` | Integer | `int(7.9)` | 7 (decimal part removed, not rounded) |
| `float()` | Float | `float("65.5")` | 65.5 |
| `str()` | String | `str(250)` | "250" |
| `bool()` | Boolean | `bool(0)` | False |

**PYTHON → OUTPUT shown in green**

```
print(int(7.9))        # truncates, does not round
print(int(-7.9))
print(int("250") + 50)
print(float("65.5") * 2)
print(float(3))
print(str(250) + " mg")
print(bool(0), bool(25), bool(""), bool("Hi"))
```

```
7
-7
300
131.0
3.0
250 mg
False True False True
```

### Truthiness rules for bool()

- **False** for: 0, 0.0, empty string "", None, and empty collections.
- **True** for every other value, including negative numbers and the string "False" (a non-empty string).

## 5.3 Errors in Type Casting

The value must be suitable for the target type, otherwise a `ValueError` occurs. Also, `int()` cannot convert a decimal string directly.

**PYTHON → OUTPUT shown in green**

```
try:
    print(int("abc"))
except ValueError as e:
    print("ValueError:", e)

try:
    print(int("12.5"))
except ValueError as e:
    print("ValueError:", e)

print(int(float("12.5")))     # correct two-step way
```

```
ValueError: invalid literal for int() with base 10: 'abc'
ValueError: invalid literal for int() with base 10: '12.5'
12
```

## 5.4 Why Type Casting Matters: the String + Number Problem

**PYTHON → OUTPUT shown in green**

```
dose = 500
unit = "mg"
# print("Dose is " + dose)        # TypeError: can only concatenate str to str
print("Dose is " + str(dose) + " " + unit)   # correct
print("Dose is", dose, unit)                 # simpler: print() converts for us
```

```
Dose is 500 mg
Dose is 500 mg
```

**EXAM TIP**

A very common 2-mark question: "What is the output of int(5.99)?" Answer: **5**. And "What does input() return?" Answer: **always a string**.

# Chapter 6: Operators

**DEFINITION - Operator**

An **operator** is a symbol that tells the interpreter to perform a particular operation (arithmetic, comparison, logical etc.) on values called **operands**. In `10 + 5`, + is the operator and 10 and 5 are the operands.

## 6.1 Arithmetic Operators

Arithmetic operators perform mathematical calculations.

| Operator | Name | Example | Result |
|---|---|---|---|
| + | Addition | `10 + 3` | 13 |
| - | Subtraction | `10 - 3` | 7 |
| * | Multiplication | `10 * 3` | 30 |
| / | Division (always float) | `10 / 3` | 3.3333333333333335 |
| // | Floor division (quotient) | `10 // 3` | 3 |
| % | Modulus (remainder) | `10 % 3` | 1 |
| ** | Exponent (power) | `10 ** 3` | 1000 |

**PYTHON → OUTPUT shown in green**

```
a, b = 17, 5
print("a + b  =", a + b)
print("a - b  =", a - b)
print("a * b  =", a * b)
print("a / b  =", a / b)
print("a // b =", a // b)
print("a % b  =", a % b)
print("a ** b =", a ** b)
print(-17 // 5, -17 % 5)
```

```
a + b  = 22
a - b  = 12
a * b  = 85
a / b  = 3.4
a // b = 3
a % b  = 2
a ** b = 1419857
-4 3
```

**REMEMBER - Floor division with negatives**

`//` rounds **down** toward negative infinity, so `-17 // 5` is -4 (not -3), and `-17 % 5` is 3. The relation `a == (a // b) * b + a % b` always holds.

### Operator precedence (BODMAS in Python)

When an expression has many operators, Python evaluates in a fixed order of priority (highest first). Operators of equal priority are evaluated left to right (except **, which goes right to left).

| Priority | Operator(s) | Description |
|---|---|---|
| 1 (highest) | ( ) | Brackets |
| 2 | ** | Exponent |

| Priority | Operator(s) | Description |
|---|---|---|
| 3 | +x, -x | Unary plus / minus |
| 4 | * / // % | Multiplication, division, floor division, modulus |
| 5 | + - | Addition, subtraction |
| 6 | == != > < >= <= | Comparison |
| 7 | not | Logical NOT |
| 8 | and | Logical AND |
| 9 (lowest) | or | Logical OR |

**PYTHON → OUTPUT shown in green**

```
print(2 + 3 * 4)        # multiplication first
print((2 + 3) * 4)      # brackets first
print(2 ** 3 ** 2)      # right to left: 2 ** 9
print(10 - 4 - 3)       # left to right
```

```
14
20
512
3
```

## 6.2 Assignment Operators

The simple assignment operator is =. Compound assignment operators combine an operation with assignment and make code shorter.

| Operator | Example | Same as |
|---|---|---|
| = | x = 5 | x gets the value 5 |
| += | x += 3 | x = x + 3 |
| -= | x -= 3 | x = x - 3 |
| *= | x *= 3 | x = x * 3 |
| /= | x /= 3 | x = x / 3 |
| //= %= **= | x //= 3 | x = x // 3 (and so on) |

**PYTHON → OUTPUT shown in green**

```
stock = 100
stock -= 25      # 25 tablets sold
stock += 60      # new batch received
stock *= 2
print(stock)
```

```
270
```

## 6.3 Comparison (Relational) Operators

Comparison operators compare two values and always return a **boolean** (True or False).

| Operator | Meaning | Example (a = 10, b = 20) | Result |
|---|---|---|---|
| == | Equal to | a == b | False |
| != | Not equal to | a != b | True |
| > | Greater than | a > b | False |
| < | Less than | a < b | True |

| Operator | Meaning | Example (a = 10, b = 20) | Result |
|---|---|---|---|
| >= | Greater than or equal to | a >= 10 | True |
| <= | Less than or equal to | b <= 15 | False |

**PYTHON → OUTPUT shown in green**

```
a, b = 10, 20
print(a == b, a != b, a > b, a < b, a >= 10, b <= 15)
print("Aspirin" == "aspirin")      # case matters
print("Aspirin".lower() == "aspirin")
print(1 < 5 < 10)                  # chained comparison
```

```
False True False True True False
False
True
True
```

**COMMON MISTAKE - = versus ==**

= **assigns** a value; == **compares** two values. Writing `if x = 5` instead of `if x == 5` is a very common mistake.

## 6.4 Logical Operators

Logical operators combine two or more conditions (boolean expressions).

| Operator | Rule | Example |
|---|---|---|
| and | True only if **both** conditions are True | age >= 18 and weight > 40 |
| or | True if **at least one** condition is True | fever or cough |
| not | Reverses the result | not is_expired |

### Truth tables

| A | B | A and B | A or B | not A |
|---|---|---|---|---|
| True | True | True | True | False |
| True | False | False | True | False |
| False | True | False | True | True |
| False | False | False | False | True |

**PYTHON → OUTPUT shown in green**

```
age, weight = 25, 55
adult = age >= 18
heavy_enough = weight >= 40
print(adult and heavy_enough)
print(adult or weight < 10)
print(not adult)

# Eligibility check for a clinical trial
eligible = (18 <= age <= 60) and (weight >= 50)
print("Eligible:", eligible)
```

```
True
True
False
Eligible: True
```

### Short-circuit evaluation

In `A and B`, if A is False, Python does not even check B (the answer must be False). In `A or B`, if A is True, B is not checked. This is called **short-circuiting** and it also prevents errors such as division by zero:

**PYTHON → OUTPUT shown in green**

```
count = 0
total = 100
print(count != 0 and total / count > 5)   # no ZeroDivisionError
```

```
False
```

## 6.5 Other Operators (for awareness)

| Type | Operators | Example |
|---|---|---|
| Membership | in, not in | "cin" in "Aspirin" gives True |
| Identity | is, is not | a is b checks if both names point to the same object |
| Bitwise | &, \|, ^, ~, <<, >> | Work on binary digits (not needed in this unit) |

## 6.6 Pharmaceutical Calculations Using Operators

**PYTHON → OUTPUT shown in green**

```
# 1. Dose by body weight: 15 mg/kg for a 60 kg patient
weight = 60
dose_per_kg = 15
total_dose = weight * dose_per_kg
print("Total dose:", total_dose, "mg")

# 2. Tablets needed if each tablet is 250 mg
tablet_strength = 250
tablets = total_dose / tablet_strength
print("Tablets:", tablets)

# 3. Dilution: C1 * V1 = C2 * V2  (find V1)
c1, c2, v2 = 10, 2, 50      # % w/v, % w/v, mL
v1 = (c2 * v2) / c1
print("Stock solution needed:", v1, "mL")

# 4. Half-life: amount left after n half-lives
amount, n = 800, 3
print("Left after 3 half-lives:", amount / 2 ** n, "mg")

# 5. Strips from tablets using // and %
tablets_total = 237
per_strip = 10
print("Full strips:", tablets_total // per_strip, "Loose tablets:", tablets_total % per_strip)
```

```
Total dose: 900 mg
Tablets: 3.6
Stock solution needed: 10.0 mL
Left after 3 half-lives: 100.0 mg
Full strips: 23 Loose tablets: 7
```

**PHARMA CONNECT - Use of // and %**

Floor division and modulus are perfect for pack-size problems: `//` gives the number of full strips or bottles and `%` gives the leftover tablets.

# Chapter 7: Input and Output Operations

A useful program communicates with the user: it takes **input** (data entered by the user) and produces **output** (results displayed). Python uses `input()` for input and `print()` for output.




*Fig. 7.1: Flow of input, casting and output*

## 7.1 Output using print()

Syntax: `print(value1, value2, ..., sep=" ", end="\n")`. Multiple values are separated by a space by default. sep changes the separator and end changes what is printed at the end (by default a new line).

**PYTHON → OUTPUT shown in green**

```
print("Paracetamol", 500, "mg")
print("Paracetamol", 500, "mg", sep="-")
print("Dose:", end=" ")
print(500)
print("A", "B", "C", sep="")
print()                  # blank line
print("Line 1\nLine 2")   # \n = new line
print("Name\tDose")       # \t = tab
print("Name\tDose\nAspirin\t75")
```

```
Paracetamol 500 mg
Paracetamol-500-mg
Dose: 500
ABC

Line 1
Line 2
Name□Dose
Name□Dose
Aspirin□75
```

### Escape sequences

| Sequence | Meaning | Sequence | Meaning |
|---|---|---|---|
| `\n` | New line | `\\` | Backslash |
| `\t` | Tab (horizontal space) | `\'` | Single quote |
| `\"` | Double quote | `\r` | Carriage return |

## 7.2 Formatted Output

Python offers three ways to put variable values inside text. The modern and recommended one is the **f-string** (formatted string literal), written with the letter `f` before the quotes and variables inside curly braces.

**PYTHON → OUTPUT shown in green**

```
name = "Metformin"
dose = 500
price = 3.4567

# 1. f-string (recommended)
print(f"{name} tablet contains {dose} mg")

# 2. format() method
print("{} tablet contains {} mg".format(name, dose))

# 3. % operator (old style)
print("%s tablet contains %d mg" % (name, dose))

# Number formatting inside f-strings
print(f"Price: Rs. {price:.2f}")          # 2 decimal places
print(f"Dose: {dose:6d}|")                # width 6
print(f"Fraction: {0.256:.1%}")           # percentage
print(f"Big number: {1234567:,}")         # thousand separator
```

```
Metformin tablet contains 500 mg
Metformin tablet contains 500 mg
Metformin tablet contains 500 mg
Price: Rs. 3.46
Dose:    500|
Fraction: 25.6%
Big number: 1,234,567
```

| Format code | Meaning | Example |
|---|---|---|
| `:.2f` | Float with 2 decimal places | `f"{3.14159:.2f}" gives 3.14` |
| `:d` | Integer | `f"{25:d}"` |
| `:10` | Minimum width 10 characters | `f"{name:10}"` |
| `:<10 :>10 :^10` | Left, right, centre aligned | `f"{name:^10}"` |
| `:,` | Thousands separator | `f"{1250000:,}" gives 1,250,000` |
| `:.1%` | Percentage | `f"{0.256:.1%}" gives 25.6%` |

A neat table can be printed using alignment codes:

**PYTHON → OUTPUT shown in green**

```
print(f"{'Drug':<14}{'Dose(mg)':>10}{'Price':>10}")
print("-" * 34)
print(f"{'Paracetamol':<14}{500:>10}{2.5:>10.2f}")
print(f"{'Amoxicillin':<14}{250:>10}{8.75:>10.2f}")
```

```
Drug           Dose(mg)     Price
----------------------------------
Paracetamol          500      2.50
Amoxicillin          250      8.75
```

## 7.3 Input using input()

The `input()` function pauses the program, shows an optional message (prompt), waits for the user to type and press Enter, and returns what was typed **always as a string**.

**PYTHON → OUTPUT shown in green**

```
name = input("Enter patient name: ")
print("Hello,", name)
print(type(name))
```

```
Enter patient name: Anita
Hello, Anita
<class 'str'>
```

The output above shows the program after the user typed *Anita*. Because the input is a string, we must convert it before doing arithmetic:

**PYTHON → OUTPUT shown in green**

```
weight = float(input("Enter weight in kg: "))
dose_per_kg = 10
total = weight * dose_per_kg
print("Total dose =", total, "mg")
```

```
Enter weight in kg: 62.5
Total dose = 625.0 mg
```

**COMMON MISTAKE - Forgetting to convert**

If we write `weight = input("Weight: ")` and then `weight * 10`, Python does not give a number: it repeats the string ten times! (For example "62" * 3 gives "626262").

**PYTHON → OUTPUT shown in green**

```
w = "62"
print(w * 3)
print(int(w) * 3)
```

```
626262
186
```

### Taking several inputs in one line

**PYTHON → OUTPUT shown in green**

```
# User types: 70 170
weight, height = input("Weight(kg) and height(cm): ").split()
weight = float(weight)
height = float(height) / 100       # cm to m
bmi = weight / height ** 2
print(f"BMI = {bmi:.2f}")
```

```
Weight(kg) and height(cm): 70 170
BMI = 24.22
```

**PYTHON → OUTPUT shown in green**

```
# Another way: map()
a, b = map(int, input("Two numbers: ").split())
print("Sum =", a + b)
```

```
Two numbers: 12 30
Sum = 42
```

## 7.4 A Complete Input-Process-Output Program

**PYTHON → OUTPUT shown in green**

```
# Pediatric dose using Clark's rule
adult_dose = float(input("Adult dose (mg): "))
child_wt = float(input("Child weight (kg): "))
child_dose = (child_wt / 70) * adult_dose
print(f"Child dose = {child_dose:.1f} mg")
```

```
Adult dose (mg): 500
Child weight (kg): 21
Child dose = 150.0 mg
```

**EXAM TIP**

Remember the three steps of every simple program: **Input → Process → Output**. Marks are given for correct casting of the input and a neat formatted output.

# Chapter 8: Basic String Operations and Manipulation

A **string** is a sequence of characters. Drug names, patient names, batch numbers, prescriptions and ADR descriptions are all text, so string handling is a core skill. Strings are **immutable**: their characters cannot be changed in place.

## 8.1 Creating Strings

**PYTHON → OUTPUT shown in green**

```
s1 = 'Aspirin'
s2 = "Pharmacist's Guide"      # double quotes allow an apostrophe
s3 = 'He said "Take after food"'
s4 = """Multi-line
string"""
s5 = ""                         # empty string
print(s1, s2, s3, sep=" | ")
print(s4)
print(len(s5))
```

```
Aspirin | Pharmacist's Guide | He said "Take after food"
Multi-line
string
0
```

## 8.2 Indexing

Every character has a position number called an **index**. Indexing starts from **0** for the first character. **Negative indexing** starts from the end (-1 is the last character).

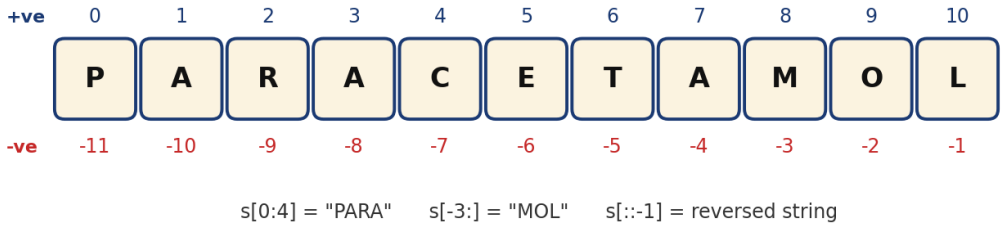



*Fig. 8.1: Positive and negative indexing of the string "PARACETAMOL"*

**PYTHON → OUTPUT shown in green**

```
drug = "PARACETAMOL"
print(drug[0])
print(drug[3])
print(drug[-1])
print(drug[-3])
print(len(drug))
# print(drug[20])   would give IndexError: string index out of range
```

```
P
A
L
M
11
```

## 8.3 Slicing

Slicing extracts a part of a string. Syntax: `string[start : stop : step]`. The **start** index is included and the **stop** index is **excluded**. Missing start means the beginning, missing stop means the end, and the default step is 1.

**PYTHON → OUTPUT shown in green**

```
drug = "PARACETAMOL"
print(drug[0:4])      # PARA
print(drug[:4])       # same
print(drug[4:])       # to the end
print(drug[-3:])      # last 3 characters
print(drug[2:9:2])    # every 2nd character
print(drug[::-1])     # reversed string
print(drug[:])        # full copy
```

```
PARA
PARA
CETAMOL
MOL
RCTM
LOMATECARAP
PARACETAMOL
```

## 8.4 Concatenation, Repetition and Membership

**PYTHON → OUTPUT shown in green**

```
a = "Anti"
b = "biotic"
print(a + b)             # concatenation
print(a + " " + b)
print("-" * 20)          # repetition
print("cet" in "PARACETAMOL".lower())   # membership
print("xyz" not in "Aspirin")
print("apple" < "banana")               # alphabetical comparison
```

```
Antibiotic
Anti biotic
--------------------
True
True
True
```

## 8.5 Immutability

**PYTHON → OUTPUT shown in green**

```
drug = "Aspirin"
try:
    drug[0] = "E"
except TypeError as e:
    print("TypeError:", e)

drug = "E" + drug[1:]    # create a new string instead
print(drug)
```

```
TypeError: 'str' object does not support item assignment
Espirin
```

## 8.6 Important String Methods

A **method** is a function that belongs to an object and is called with a dot: `string.method()`. String methods do not change the original string; they **return a new string**.

| Method | Purpose | Example | Result |
|---|---|---|---|
| `upper()` | All capitals | `"aspirin".upper()` | `ASPIRIN` |
| `lower()` | All small letters | `"ASPIRIN".lower()` | `aspirin` |
| `title()` | First letter of each word capital | `"cough syrup".title()` | `Cough Syrup` |
| `capitalize()` | First letter of string capital | `"aspirin tab".capitalize()` | `Aspirin tab` |
| `swapcase()` | Reverse the case | `"Hello".swapcase()` | `hELLO` |
| `strip()` | Remove spaces from both ends | `" tab ".strip()` | `tab` |
| `lstrip()` / `rstrip()` | Remove spaces from left / right | `" tab".lstrip()` | `tab` |
| `replace(old, new)` | Replace text | `"mg".replace("mg","g")` | `g` |
| `find(sub)` | Index of first match, -1 if absent | `"aspirin".find("pi")` | `3` |
| `index(sub)` | Like find but error if absent | `"aspirin".index("s")` | `1` |
| `count(sub)` | Number of occurrences | `"banana".count("a")` | `3` |
| `startswith(x)` | True if begins with x | `"Tab 5mg".startswith("Tab")` | `True` |
| `endswith(x)` | True if ends with x | `"file.csv".endswith(".csv")` | `True` |
| `split(sep)` | Break into a list | `"a,b,c".split(",")` | `['a', 'b', 'c']` |
| `join(list)` | Join list items with a separator | `"-".join(["A","B"])` | `A-B` |
| `isdigit()` | True if all characters are digits | `"500".isdigit()` | `True` |
| `isalpha()` | True if all are letters | `"Aspirin".isalpha()` | `True` |
| `isalnum()` | True if letters/digits only | `"Batch1".isalnum()` | `True` |
| `islower()` / `isupper()` | Check the case | `"abc".islower()` | `True` |
| `zfill(n)` | Pad with zeros on the left | `"45".zfill(5)` | `00045` |

**PYTHON → OUTPUT shown in green**

```
text = "   paracetamol 500 MG tablet   "
clean = text.strip()
print(clean)
print(clean.upper())
print(clean.title())
print(clean.replace("MG", "mg"))
print(clean.split())
print(clean.count("a"))
print(clean.find("500"))
print("-".join(["BP101T", "Unit", "I"]))
```

```
paracetamol 500 MG tablet
PARACETAMOL 500 MG TABLET
Paracetamol 500 Mg Tablet
paracetamol 500 mg tablet
['paracetamol', '500', 'MG', 'tablet']
4
12
BP101T-Unit-I
```

## 8.7 Useful Built-in Functions with Strings

**PYTHON → OUTPUT shown in green**

```
s = "Pharmacy"
print(len(s))          # number of characters
print(max(s), min(s))  # by character order
print(sorted(s))       # list of sorted characters
print(str(123) + "4")
print(ord("A"), chr(66))   # character codes
```

```
8
y P
['P', 'a', 'a', 'c', 'h', 'm', 'r', 'y']
1234
65 B
```

## 8.8 Strings in Practice: Pharmaceutical Examples

**PYTHON → OUTPUT shown in green**

```
# 1. Parse a drug label
label = "Paracetamol 500 mg"
parts = label.split()
name, strength, unit = parts[0], int(parts[1]), parts[2]
print(name, strength * 2, unit)

# 2. Check for a palindrome
word = "madam"
print(word == word[::-1])

# 3. Count vowels in a drug name
drug = "Amoxicillin"
vowels = sum(1 for ch in drug.lower() if ch in "aeiou")
print("Vowels:", vowels)

# 4. Make a batch code
batch = "BN" + "7".zfill(4) + "-" + "2026"
print(batch)

# 5. Extract the abbreviation (first letters)
phrase = "adverse drug reaction"
abbr = "".join(w[0] for w in phrase.split()).upper()
print(abbr)
```

```
Paracetamol 1000 mg
True
Vowels: 5
BN0007-2026
ADR
```

## 8.9 Raw Strings and Special Cases

**PYTHON → OUTPUT shown in green**

```
path = "C:\\data\\pk_study.csv"      # backslash escaped
raw = r"C:\data\pk_study.csv"       # raw string: backslash taken literally
print(path)
print(raw)
```

```
C:\data\pk_study.csv
C:\data\pk_study.csv
```

**EXAM TIP - Typical exam questions**

(a) Write a program to reverse a string. (b) Count vowels. (c) Check a palindrome. (d) Explain slicing with examples. (e) Explain any five string methods. Prepare all five; the table in 8.6 gives ready material.

# Chapter 9: Standard and Third-Party Libraries

## 9.1 Modules, Packages and Libraries

- A **module** is a single Python file (`.py`) containing functions, variables and classes that can be reused in other programs.
- A **package** is a folder of related modules.
- A **library** is a collection of modules and packages that together provide a set of related functions (for example NumPy or Pandas).

Libraries save time. Instead of writing code to compute a square root or draw a graph, we **import** the code written by experts.

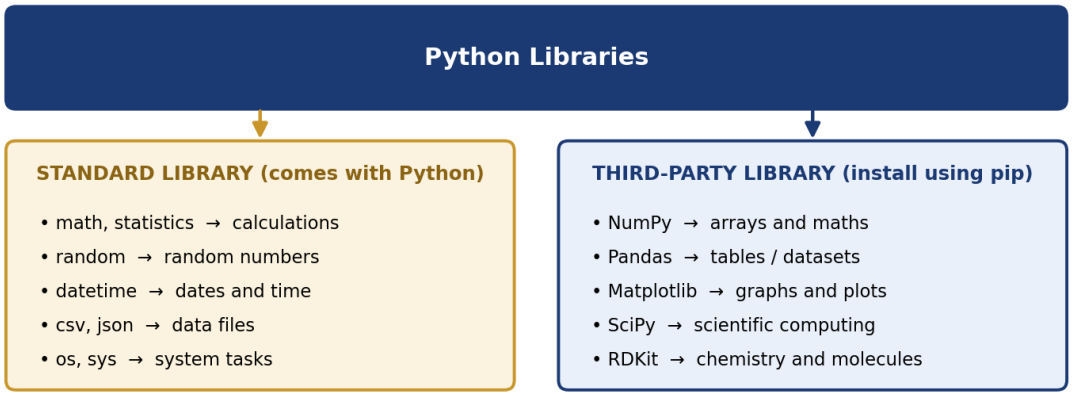



*Fig. 9.1: Standard versus third-party libraries*

## 9.2 Standard Libraries

The **Python Standard Library** is installed along with Python itself; no extra download is needed. It is often said that Python comes "with batteries included".

| Library | Use | Example functions |
|---|---|---|
| `math` | Mathematical functions | sqrt(), pow(), log(), exp(), ceil(), floor(), pi |
| `statistics` | Basic statistics | mean(), median(), mode(), stdev() |
| `random` | Random numbers | random(), randint(), choice(), shuffle() |
| `datetime` | Dates and times | datetime.now(), date(), timedelta() |
| `csv` | Read and write CSV files | reader(), writer() |
| `json` | Work with JSON data | load(), dump(), loads(), dumps() |
| `os / sys` | Operating system and interpreter | os.getcwd(), os.listdir(), sys.version |
| `string` | String constants | ascii_letters, digits |

### Ways of importing

**PYTHON → OUTPUT shown in green**

```
import math                   # style 1: import the whole module
print(math.sqrt(144))

from math import sqrt, pi      # style 2: import selected names
print(sqrt(81), round(pi, 4))

import math as m               # style 3: use a short alias
print(m.pow(2, 5))

from math import *             # style 4: import all (not recommended)
```

```
12.0
9.0 3.1416
32.0
```

**PYTHON → OUTPUT shown in green**

```
import math
print(math.sqrt(2))
print(math.ceil(4.2), math.floor(4.8))
print(round(math.log(2) / 6, 4))      # elimination rate constant for t1/2 = 6 h
print(math.exp(-0.1155 * 12))
print(math.factorial(5))
print(math.pi)
```

```
1.4142135623730951
5 4
0.1155
0.2500736011120941
120
3.141592653589793
```

**PYTHON → OUTPUT shown in green**

```
import statistics as st
assay = [98.5, 99.2, 101.0, 100.4, 99.8]     # % label claim
print("Mean   :", st.mean(assay))
print("Median :", st.median(assay))
print("Std dev:", round(st.stdev(assay), 3))
```

```
Mean   : 99.78
Median : 99.8
Std dev: 0.981
```

**PYTHON → OUTPUT shown in green**

```
import random
random.seed(7)                         # same results every run
print(random.randint(1, 10))
print(random.choice(["Tab A", "Tab B", "Tab C"]))
print(round(random.random(), 3))
```

```
6
Tab A
0.395
```

**PYTHON → OUTPUT shown in green**

```
from datetime import date, timedelta
mfg = date(2026, 1, 15)
expiry = mfg + timedelta(days=730)    # 2 years shelf life
print("Manufactured:", mfg)
print("Expiry      :", expiry)
print("Weekday     :", expiry.strftime("%A"))
```

```
Manufactured: 2026-01-15
Expiry      : 2028-01-15
Weekday     : Saturday
```

## 9.3 Third-Party Libraries

Third-party libraries are written by the community or companies and are **not** included with Python. They must be **installed** before use. They are published on the **Python Package Index (PyPI, pypi.org)**, which hosts hundreds of thousands of packages.

| Library | Purpose | Relevance to pharmacy |
|---|---|---|
| `NumPy` | Fast numerical arrays and mathematics | Calculations on large sets of PK or assay values (Unit III) |
| `Pandas` | Tables (DataFrames), cleaning, analysis | ADR reports, PK datasets, sales data (Unit IV) |
| `Matplotlib` | Graphs and plots | Concentration-time curves, dissolution profiles (Unit V) |
| `SciPy` | Scientific and statistical computing | Curve fitting, t-test, integration (AUC) |
| `scikit-learn` | Machine learning | Disease prediction, classification of compounds |
| `RDKit` | Cheminformatics | Molecular structures, descriptors, drug design |
| `Biopython` | Bioinformatics | Protein and gene sequences |
| `openpyxl` | Read/write Excel files | Reports from Excel sheets |

## 9.4 Installing and Uninstalling Libraries with pip

**pip** (Pip Installs Packages) is the package manager of Python. It downloads libraries from PyPI and installs them. It comes automatically with Python 3. An **internet connection** is required. Commands are typed in the **terminal / Command Prompt**, not inside the Python prompt.

| Task | Command |
|---|---|
| Install a library | `pip install numpy` |
| Install several libraries | `pip install numpy pandas matplotlib` |
| Install a particular version | `pip install pandas==2.2.0` |
| Upgrade a library | `pip install --upgrade numpy` |
| Uninstall a library | `pip uninstall numpy` |
| List all installed libraries | `pip list` |
| Show details of a library | `pip show numpy` |
| Save the list of libraries | `pip freeze > requirements.txt` |
| Install from that list | `pip install -r requirements.txt` |
| Check pip version | `pip --version` |

**TERMINAL / COMMAND PROMPT**

```
pip install numpy pandas matplotlib
pip show numpy
pip uninstall numpy
pip list
```

## Installing libraries inside IDEs

- **Jupyter Notebook / Colab:** in a code cell type `!pip install pandas` (or `%pip install pandas`).
- **PyCharm:** File → Settings → Project → Python Interpreter → "+" button → search the package → Install Package.
- **VS Code:** open the integrated terminal (Ctrl + `) and use the pip commands.
- **Anaconda:** `conda install numpy` can also be used.

## Using an installed library

**PYTHON → OUTPUT shown in green**

```
import numpy as np          # np is the standard alias
marks = np.array([45, 38, 50, 42])
print(marks.mean())
print(np.__name__)
```

```
43.75
numpy
```

## Virtual environments (introduction)

Different projects may need different versions of the same library. A **virtual environment** is an isolated Python workspace with its own libraries, so that one project does not disturb another.

**TERMINAL / COMMAND PROMPT**

```
python -m venv pharma_env            # create
pharma_env\Scripts\activate           # activate (Windows)
source pharma_env/bin/activate       # activate (Linux/macOS)
pip install pandas                   # installs only inside this environment
deactivate                           # leave the environment
```

## Common installation problems

| Error message | Cause | Remedy |
|---|---|---|
| 'pip' is not recognized | Python not added to PATH | Re-install Python and tick "Add to PATH", or use: python -m pip install ... |
| ModuleNotFoundError: No module named "x" | Library not installed (or installed in a different Python) | Run pip install x in the same environment as your IDE |
| Could not find a version / connection error | No internet or wrong spelling | Check the connection and the package name |
| Permission denied | No rights to install | Use pip install --user x or a virtual environment |

**EXAM TIP - Likely question**

Q. Differentiate standard and third-party libraries. How are libraries installed and uninstalled? → Give a comparison table (included with Python vs installed with pip), 3 examples each, and the commands `pip install` and `pip uninstall` with a short example.

| Standard library | Third-party library |
|---|---|
| Comes pre-installed with Python | Must be installed separately with pip |

| Standard library | Third-party library |
|---|---|
| Examples: math, random, datetime, csv | Examples: NumPy, Pandas, Matplotlib |
| No internet needed | Internet needed to install |
| Maintained by the Python core team | Maintained by the community or companies |
| Only import is needed | Install first, then import |

# Chapter 10: Integrated Pharmaceutical Programs

The programs below combine everything from Unit I: variables, data types, casting, operators, input/output, strings and a library. Type and run each one yourself; then change the numbers and see what happens.

## Program 1: BMI Calculator (Body Mass Index)

Formula: BMI = weight (kg) / height $(m)^2$. WHO classification: below 18.5 underweight, 18.5-24.9 normal, 25-29.9 overweight, 30 and above obese. (Classification uses decisions, covered in Unit II; here we only calculate and print.)

**PYTHON → OUTPUT shown in green**

```
name = input("Patient name: ")
weight = float(input("Weight (kg): "))
height_cm = float(input("Height (cm): "))

height_m = height_cm / 100
bmi = weight / height_m ** 2

print("-" * 30)
print(f"Patient : {name.title()}")
print(f"BMI     : {bmi:.1f} kg/m2")
print("Normal range 18.5 - 24.9:", 18.5 <= bmi <= 24.9)
```

```
Patient name: rahul sharma
Weight (kg): 68
Height (cm): 172
------------------------------
Patient : Rahul Sharma
BMI     : 23.0 kg/m2
Normal range 18.5 - 24.9: True
```

## Program 2: Dose Calculator by Body Weight

**PYTHON → OUTPUT shown in green**

```
drug = "Amoxicillin"
weight = float(input("Child weight (kg): "))
dose_per_kg_per_day = 25        # mg/kg/day
doses_per_day = 3
syrup_strength = 125            # mg per 5 mL

daily_dose = weight * dose_per_kg_per_day
single_dose = daily_dose / doses_per_day
volume_ml = single_dose / syrup_strength * 5

print(f"Drug         : {drug}")
print(f"Daily dose   : {daily_dose:.0f} mg")
print(f"Single dose  : {single_dose:.1f} mg")
print(f"Syrup volume : {volume_ml:.1f} mL, {doses_per_day} times a day")
```

```
Child weight (kg): 15
Drug         : Amoxicillin
Daily dose   : 375 mg
Single dose  : 125.0 mg
Syrup volume : 5.0 mL, 3 times a day
```

## Program 3: Creatinine Clearance (Cockcroft-Gault, male)

CrCl (mL/min) = [(140 - age) x weight] / (72 x serum creatinine). For females the result is multiplied by 0.85. This estimates kidney function and guides renal dose adjustment.

**PYTHON → OUTPUT shown in green**

```
age = int(input("Age (years): "))
weight = float(input("Weight (kg): "))
scr = float(input("Serum creatinine (mg/dL): "))

crcl = ((140 - age) * weight) / (72 * scr)
print(f"CrCl (male)   = {crcl:.1f} mL/min")
print(f"CrCl (female) = {crcl * 0.85:.1f} mL/min")
```

```
Age (years): 55
Weight (kg): 70
Serum creatinine (mg/dL): 1.1
CrCl (male)   = 75.1 mL/min
CrCl (female) = 63.9 mL/min
```

## Program 4: Drug Label Reader (strings)

**PYTHON → OUTPUT shown in green**

```
label = "  TAB. Metformin HCl 500MG | Batch: MF2406 | Qty: 30  "
label = label.strip()
parts = [p.strip() for p in label.split("|")]
product = parts[0]
batch = parts[1].split(":")[1].strip()
qty = int(parts[2].split(":")[1])

print("Product :", product.title())
print("Batch   :", batch)
print("Quantity:", qty)
print("Is tablet?", product.upper().startswith("TAB"))
print("Strips of 10:", qty // 10)
```

```
Product : Tab. Metformin Hcl 500Mg
Batch   : MF2406
Quantity: 30
Is tablet? True
Strips of 10: 3
```

## Program 5: Expiry Date Calculator (library use)

**PYTHON → OUTPUT shown in green**

```
from datetime import date, timedelta

mfg = date(2026, 3, 1)
shelf_life_months = 24
expiry = date(mfg.year + shelf_life_months // 12, mfg.month, 1) - timedelta(days=1)
print("Mfg date :", mfg.strftime("%d-%m-%Y"))
print("Expiry   :", expiry.strftime("%b %Y"))
print("Days from mfg to expiry:", (expiry - mfg).days)
```

```
Mfg date : 01-03-2026
Expiry   : Feb 2028
Days from mfg to expiry: 730
```

## Program 6: Assay Summary using the statistics library

**PYTHON → OUTPUT shown in green**

```
import statistics as st

batch_assay = [99.1, 100.3, 98.7, 101.2, 99.9, 100.5]
mean = st.mean(batch_assay)
sd = st.stdev(batch_assay)
rsd = sd / mean * 100
print(f"Mean = {mean:.2f} %")
print(f"SD   = {sd:.3f}")
print(f"RSD  = {rsd:.2f} %")
print("Within 98.0 - 102.0 %:", min(batch_assay) >= 98 and max(batch_assay) <= 102)
```

```
Mean = 99.95 %
SD   = 0.925
RSD  = 0.93 %
Within 98.0 - 102.0 %: True
```

# Chapter 11: Unit I Summary, Glossary and Quick Revision

## 11.1 One-page Summary

- **Python** is a high-level, interpreted, open-source language created by Guido van Rossum (1991), valued for simple syntax and a huge library collection.
- **Installation:** download from python.org, tick "Add to PATH", verify with `python --version`. **IDEs** (Jupyter, PyCharm, VS Code) combine editing, running and debugging, giving auto-complete, error detection and debugging that plain text editors lack.
- **Variables** are named memory locations created by assignment; Python is dynamically typed. Basic data types: **int, float, str, bool**.
- **Type casting** converts between types: `int(), float(), str(), bool()`. Implicit casting is automatic; explicit casting is manual.
- **Operators:** arithmetic (+ - * / // % **), comparison (== != > < >= <=), logical (and, or, not) and assignment (= += -=).
- **Input/Output:** `input()` returns a string; `print()` displays output; f-strings format output neatly.
- **Strings** are immutable sequences supporting indexing, slicing, concatenation, repetition and many methods (upper, lower, strip, replace, split, join, find, count).
- **Libraries:** standard (math, random, datetime, statistics) come with Python; third-party (NumPy, Pandas, Matplotlib) are installed with `pip install` and removed with `pip uninstall`.

## 11.2 Quick Reference Table

| Topic | Key syntax | Remember |
|---|---|---|
| Comment | `# text` | Ignored by Python |
| Type check | `type(x)` | Returns the class of x |
| Cast | `int("5")` | int(5.9) = 5 |
| Division | `7 / 2, 7 // 2, 7 % 2` | 3.5, 3, 1 |
| Power | `2 ** 3` | 8 |
| Input number | `float(input("..."))` | input is always str |
| f-string | `f"{x:.2f}"` | 2 decimal places |
| Slice | `s[a:b:c]` | b is excluded |
| Reverse | `s[::-1]` | Step -1 |
| Import | `import math` | math.sqrt(4) |
| Install | `pip install pandas` | Run in terminal |
| Uninstall | `pip uninstall pandas` | Asks for confirmation |

## 11.3 Glossary

| Term | Meaning |
|---|---|
| Algorithm | A step-by-step procedure to solve a problem |
| Argument | A value passed to a function, e.g., "Hi" in print("Hi") |
| Bytecode | Intermediate code produced from source code and run by the PVM |

| Term | Meaning |
|---|---|
| Casting | Converting one data type to another |
| Comment | Explanatory text ignored by the interpreter |
| Debugging | Finding and removing errors in a program |
| Escape sequence | Backslash code such as \n for a special character |
| f-string | String prefixed with f that inserts values using {} |
| IDE | Integrated Development Environment: editor + run + debug tools |
| Immutable | Cannot be changed after creation (str, int, float, bool) |
| Index | Position number of an item, starting from 0 |
| Interpreter | Program that translates and executes code line by line |
| Kernel | Background engine that executes code in a Jupyter notebook |
| Library | Collection of ready-made modules and packages |
| Module | A single .py file of reusable code |
| Operand | Value on which an operator acts |
| PyPI | Python Package Index: online store of third-party packages |
| REPL | Read-Evaluate-Print Loop: the interactive Python prompt |
| Slicing | Extracting a part of a sequence using [start:stop:step] |
| Syntax | The grammar rules of a programming language |
| Variable | Named storage location holding a value |
| Virtual environment | Isolated workspace with its own libraries |

# Chapter 12: Exam Practice Section

Marks pattern for BP101T: total 50 (Sessional Evaluation 20 + End Semester Examination 30). Practise with the question types below. Answers to MCQs are given at the end of this section.

## 12.1 Multiple Choice Questions (30)

**Q1.** Who created Python? (a) Dennis Ritchie (b) Guido van Rossum (c) James Gosling (d) Bjarne Stroustrup

**Q2.** Python is a/an _____ language. (a) compiled only (b) machine (c) interpreted, high-level (d) assembly

**Q3.** Which option must be ticked while installing Python on Windows? (a) Install for all (b) Add python.exe to PATH (c) Disable length limit (d) Create shortcuts

**Q4.** Which of the following is an IDE? (a) Notepad (b) PyCharm (c) MS Paint (d) Chrome

**Q5.** The file extension of a Jupyter notebook is (a) .py (b) .txt (c) .ipynb (d) .jup

**Q6.** Which is NOT an advantage of an IDE? (a) Debugger (b) Code completion (c) Syntax highlighting (d) Smaller size than Notepad

**Q7.** Which is a valid variable name? (a) 2dose (b) dose-mg (c) dose_mg (d) class

**Q8.** The data type of 12.0 is (a) int (b) float (c) str (d) bool

**Q9.** Output of type(True) is (a) <class 'int'> (b) <class 'bool'> (c) <class 'str'> (d) <class 'float'>

**Q10.** Output of int(7.9) is (a) 8 (b) 7 (c) 7.9 (d) Error

**Q11.** input() always returns (a) int (b) float (c) str (d) bool

**Q12.** Which function converts a string to a decimal number? (a) int() (b) float() (c) str() (d) bool()

**Q13.** Output of 17 // 5 is (a) 3.4 (b) 3 (c) 2 (d) 4

**Q14.** Output of 17 % 5 is (a) 3 (b) 2 (c) 3.4 (d) 0

**Q15.** Output of 2 ** 4 is (a) 8 (b) 6 (c) 16 (d) 24

**Q16.** Which operator is used for comparison of equality? (a) = (b) == (c) != (d) =>

**Q17.** Output of not (5 > 3) (a) True (b) False (c) 5 (d) Error

**Q18.** Output of True + True (a) True (b) 2 (c) Error (d) 1

**Q19.** Which correctly prints without a new line at the end? (a) print("a", end="") (b) print("a", stop="") (c) print("a", new="") (d) printn("a")

**Q20.** The first index in a Python string is (a) 1 (b) -1 (c) 0 (d) None

**Q21.** For s = "PHARMACY", s[-1] is (a) P (b) Y (c) C (d) Error

**Q22.** For s = "PHARMACY", s[0:3] is (a) PHA (b) PHAR (c) HAR (d) HA

**Q23.** Which method removes leading and trailing spaces? (a) trim() (b) strip() (c) cut() (d) remove()

**Q24.** "a,b,c".split(",") returns (a) a string (b) a list (c) an integer (d) a boolean

**Q25.** Strings in Python are (a) mutable (b) immutable (c) not iterable (d) always numeric

**Q26.** Which library comes with Python (standard library)? (a) NumPy (b) Pandas (c) math (d) Matplotlib

**Q27.** Which command installs a library? (a) pip get numpy (b) pip install numpy (c) python numpy (d) install pip numpy

**Q28.** Which command removes a library? (a) pip remove numpy (b) pip delete numpy (c) pip uninstall numpy (d) pip erase numpy

**Q29.** PyPI stands for (a) Python Package Index (b) Python Private Interface (c) Primary Python Installer (d) Python Program Interpreter

**Q30.** import numpy as np means (a) np is an alias of numpy (b) numpy is deleted (c) np is installed (d) numpy becomes a string

**REMEMBER - MCQ Answer Key**

1-b, 2-c, 3-b, 4-b, 5-c, 6-d, 7-c, 8-b, 9-b, 10-b, 11-c, 12-b, 13-b, 14-b, 15-c, 16-b, 17-b, 18-b, 19-a, 20-c, 21-b, 22-a, 23-b, 24-b, 25-b, 26-c, 27-b, 28-c, 29-a, 30-a

## 12.2 Fill in the Blanks

1. Python was created by _____ in 1991.

2. The interactive Python prompt is called _____.

3. The software that combines editor, run and debug tools is called _____.

4. Variable names cannot start with a _____.

5. The function used to find the data type is _____.

6. The result of 9 // 2 is _____.

7. The boolean value of an empty string is _____.

8. input() returns data of type _____.

9. The stop index in slicing is _____ (included/excluded).

10. The method that converts text to capital letters is _____.

11. Third-party libraries are hosted on _____.

12. The command to list installed libraries is _____.

13. Escape sequence for a new line is _____.

14. The operator for power is _____.

15. Jupyter code cells are executed using _____ keys.

**REMEMBER - Answers**

1. Guido van Rossum; 2. REPL; 3. IDE; 4. digit; 5. type(); 6. 4; 7. False; 8. str; 9. excluded; 10. upper(); 11. PyPI; 12. pip list; 13. \\n; 14. **; 15. Shift + Enter

## 12.3 Short Answer Questions with Model Answers (2-3 Marks)

**Q1. Define Python. List four features.**

**Ans.** Python is a high-level, interpreted, general-purpose language created by Guido van Rossum. Features: simple and readable syntax, interpreted, dynamically typed, free and open source, portable across operating systems, and rich library support.

**Q2. What is an IDE? Name three Python IDEs.**

**Ans.** An Integrated Development Environment is software that combines a code editor, a way to run programs and a debugger in one window. Examples: Jupyter Notebook, PyCharm and VS Code (with

Python extension).

**Q3. What is a variable? State the rules for naming.**

**Ans.** A variable is a named memory location used to store data. Rules: letters, digits and underscore only; cannot begin with a digit; no spaces or special symbols; cannot be a keyword; case-sensitive.

**Q4. Differentiate between implicit and explicit type casting.**

**Ans.** Implicit casting is done automatically by Python (e.g., int + float gives float). Explicit casting is done by the programmer using functions such as int(), float(), str() and bool().

**Q5. What is the difference between / and // operators?**

**Ans.** / is true division and always returns a float (7 / 2 = 3.5). // is floor division and returns the quotient rounded down (7 // 2 = 3).

**Q6. Why do we use f-strings? Give an example.**

**Ans.** F-strings insert variable values and format numbers inside text neatly. Example: print(f"BMI = {bmi:.2f}") displays BMI with two decimal places.

**Q7. What is slicing? Explain with an example.**

**Ans.** Slicing extracts a part of a string using string[start:stop:step]; start is included and stop is excluded. For s = "PARACETAMOL", s[0:4] gives "PARA" and s[::-1] gives the reversed string.

**Q8. Why are strings called immutable?**

**Ans.** Once a string is created its characters cannot be modified in place. s[0] = "E" gives TypeError; a new string must be created instead.

**Q9. Differentiate standard library from third-party library.**

**Ans.** Standard libraries (math, random, datetime) come installed with Python and only need import. Third-party libraries (NumPy, Pandas, Matplotlib) must be installed using pip from PyPI before use.

**Q10. Write the commands to install and uninstall pandas.**

**Ans.** pip install pandas and pip uninstall pandas, typed in the command prompt/terminal (use !pip in Jupyter cells).

**Q11. What is the output of print(10 + 3 * 2 ** 2)?**

**Ans.** 22. Exponent first (2**2 = 4), then multiplication (3*4 = 12), then addition (10 + 12).

**Q12. Why must input() be converted for calculations?**

**Ans.** Because input() always returns a string. For numeric work it must be converted with int() or float(), otherwise arithmetic gives wrong results or a TypeError.

## 12.4 Long Answer Questions (5-10 Marks): Answer Outlines

### LQ1. Explain installation of Python and the advantages of IDEs over text editors. (10 marks)

- Introduction: Python, its creator and nature.
- Installation steps on Windows (download, tick PATH, install, verify) and one line on Linux/macOS.
- Definition of text editor and IDE; comparison table (components, running, auto-complete, debugging, error check).
- Any 8 advantages of IDE; short descriptions of Jupyter Notebook, PyCharm and VS Code.
- Conclusion: IDE improves productivity; choose by need.

### LQ2. Describe the data types in Python with suitable examples. Explain type casting. (10 marks)

- Variable definition and dynamic typing; naming rules.
- int, float, str, bool with example and type() output; mention None.
- Type casting: definition, implicit and explicit; table of int(), float(), str(), bool() with results.
- Errors (ValueError) and the input() example with float(input()).

### LQ3. Explain operators in Python with examples. (10 marks)

- Definition of operator and operand.
- Arithmetic operators table with results (include // % **).
- Comparison operators table; logical operators with truth table; assignment operators.
- Precedence order; one pharmacy example (dose or dilution calculation).

### LQ4. Explain string operations and any ten string methods. (10 marks)

- Definition, creation (single, double, triple quotes), immutability.
- Indexing (positive/negative), slicing with the PARACETAMOL example, concatenation, repetition, membership.
- Table of ten methods with examples (upper, lower, strip, replace, find, count, split, join, startswith, isdigit).
- A short program: reverse a string or parse a drug label.

### LQ5. Write a note on standard and third-party libraries. How are libraries installed and uninstalled? (10 marks)

- Module, package, library definitions.
- Standard library table with examples and import styles.
- Third-party libraries (NumPy, Pandas, Matplotlib, SciPy, RDKit) with relevance.
- pip commands: install, uninstall, list, show, upgrade; PyPI; virtual environment; common errors.

### LQ6. Write a Python program to calculate BMI and creatinine clearance and print neat output. (5 marks)

- Use input() with float(); formulas BMI = kg / m^2 and CrCl = ((140 - age) x wt) / (72 x SCr).
- Print with f-strings and :.2f; refer to Programs 1 and 3 of Chapter 10.

## 12.5 Predict the Output

**PYTHON → OUTPUT shown in green**

```
print(7 + 3 * 2)
print((7 + 3) * 2)
print(15 / 4, 15 // 4, 15 % 4)
print(int("8") + float("2.5"))
print("Mg" * 3)
print("Python"[1:4])
print(bool(""), bool("0"))
print(5 > 3 and 2 > 4)
print("ab" + "cd" == "abcd")
```

```
13
20
3.75 3 3
10.5
MgMgMg
yth
False True
False
True
```

## 12.6 Practice Programs

1. Take drug name and dose from the user and print "Take 500 mg of Paracetamol twice daily" using an f-string.
2. Convert body temperature from Celsius to Fahrenheit: F = C x 9/5 + 32.
3. Calculate the total bill when the user enters price per strip and number of strips (add 5% GST).
4. Input a string and print its length, upper case, lower case and reverse.
5. Count the number of vowels and consonants in a given drug name.
6. Calculate the infusion rate (mL/hr) = total volume / time in hours.
7. Calculate the number of tablets required for a course: tablets per day x number of days.
8. Use the math library to find the elimination rate constant k = 0.693 / t1/2 for t1/2 entered by the user.
9. Use datetime to display today's date and the date 30 days later (a follow-up date).
10. Make a program that asks for an ADR description and prints how many words it contains.

**REMEMBER - Final advice**

Type every program yourself instead of copying. Run it, deliberately create an error, read the message and fix it. This habit is the fastest way to learn programming and to score well in both theory and practical examinations.



# End of Unit I

Next: Unit II - Control Structures and Functions

**DRx LEARNING HUB**

Prepared by DRx Deepak Jatav

*Learn. Practise. Succeed.*

*Reference sources recommended in the syllabus: Weiss (2017) Scientific Computing for Chemists with Python; Perkovic (2015) Introduction to Computing Using Python; Sweigart (2025) Automate the Boring Stuff with Python; W3Schools Python Tutorial.*